

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Title of the master thesis

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc.

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Max Mustermann
Student ID: 111222
Date: 12.12.1212

Supervising tutor: Prof. Dr. W. Hardt
(further supervisors)

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Keywords: Keyword1, Keyword2, Keyword3, ...max 5

Contents

Contents . . . 3

List of Figures . . . 5

List of Tables . . . 7

List of Abbreviations . . . 8

1 Introduction . . . 9

2 Background and State of the Art . . . 10

3 Data Acquisition and Dataset Construction . . . 11
3.1 Sensor Platform and Computational Constraints . . . 11
3.2 Assembly Scenario and Activity Taxonomy . . . 12
3.3 Data-Collection Firmware . . . 14
3.3.1 Dual-Stream UDP Architecture . . . 14
3.3.2 Audio Double-Buffering and Packet Framing . . . 14
3.3.3 Microsecond Timestamping and Sequence Numbering . . . 15
3.3.4 Multi-Device Capture and Body Placements . . . 16
3.4 Recording Protocol . . . 16
3.5 Ground-Truth Annotation from Synchronised Video . . . 17
3.5.1 Determination of the Audio-Video Offset . . . 18
3.5.2 Interval Labels and Sample-Level Label Embedding . . . 18
3.6 Signal Conditioning . . . 19
3.6.1 Packet-Loss Detection and Audio-Dropout Reconstruction . . . 20
3.6.2 IMU Decimation to 80 Hz . . . 20
3.6.3 Audio Resampling to a Strict 16 kHz Timeline . . . 21
3.7 Window Generation and Labelling . . . 21
3.8 Dataset Statistics, Class Distribution, and Imbalance . . . 23
3.9 Data-Quality Assessment and Known Artefacts . . . 24
3.10 Chapter Summary . . . 25

4 Model Design and Training . . . 26
4.1 Overview of the Experimental Design . . . 26
4.2 Input Representations . . . 27
4.2.1 Inertial Windows . . . 27

4.2.2 Audio Log-Mel Spectrograms . . . 28
4.3 Single-Modality Models . . . 29
4.3.1 Inertial Model . . . 29
4.3.2 Audio Model and Temporal Pooling Variants . . . 29
4.4 Fusion Architectures . . . 31
4.4.1 Early Fusion . . . 31
4.4.2 Embedding-Level Fusion . . . 32
4.4.3 Decision-Level Fusion . . . 32
4.5 Classical Baseline . . . 34
4.6 Deployment-Oriented Model Variants . . . 35
4.7 Training Procedure . . . 36
4.7.1 Software and Hardware Environment . . . 36
4.7.2 Training Settings . . . 37
4.8 Evaluation Protocol . . . 38
4.8.1 Grouped Train/Validation Split . . . 38
4.8.2 Metrics . . . 39
4.9 Chapter Summary . . . 39

5 On-Device Deployment . . . 41

6 Results and Evaluation . . . 42

7 Discussion . . . 43

8 Conclusion . . . 44

Bibliography . . . 45

List of Figures

3.1 The M5StickC Plus2 wearable device. The ESP32-PICO-V3-02 system-on-chip, the six-axis IMU (MPU6886), and the microphone (SPM1423) are all integrated on the board. Image: M5Stack [5]. 12

3.2 Architecture of the multimodal data-acquisition system. Each of the four wearable devices samples an IMU and a microphone independently, timestamps and frames the samples, and streams them as two separate UDP streams (inertial on one port, audio on another) to a single host, which separates the devices by packet source address and writes raw per-device files. 15

3.3 Signal-conditioning and windowing pipeline, applied per device. The inertial and acoustic branches are brought to fixed rates before both are cut on the same window boundaries. 19

3.4 Window generation and majority-vote labelling. Two-second windows advance in one-second steps (W_1, W_2, W_3 shown stacked for clarity). A window takes the majority activity among its labelled samples; a window whose labelled coverage falls below 70 %, such as one spanning the transition region, is discarded. 22

3.5 Number of two-second windows per activity class in the dominant-wrist dataset ($n = 1380$). The distribution is dominated by the screwdriving classes; `screw_tightening` alone accounts for about 38 % of the windows and the two screwdriving classes together for more than half. 24

4.1 Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models. 28

4.2 Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries. 30

4.3 The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models. 32

4.4 Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom). 34
4.5 Grouped splitting. Overlapping windows are grouped into activity instances, and whole instances are assigned to either the training or the validation set. A random split would distribute the overlapping windows of one instance across both sets. 38

List of Tables

3.1 Activity classes in the modelling dataset. Descriptions are taken from the 32-step assembly protocol. "Typ. dur." is the range of mean coded-segment durations across the three recording sessions; for `screw_tightening` it covers both the tightening and the merged loosening segments. "Win." is the number of two-second windows per class in the final dominant-wrist dataset (Section 3.8). 13
3.2 Body placements: data recorded during acquisition versus data used as model input in the final pipeline. Only the dominant-wrist inertial and acoustic streams reach the model. 17
3.3 Data-acquisition sessions. "Session" paraphrases the folder label used in the project records. "(+1)" denotes one further sequence marked invalid and set aside. 17
3.4 Composition of the dominant-wrist modelling dataset. 23
3.5 Data-quality indicators for the dataset. 24

4.1 Overview of the evaluated models. 27
4.2 Input representations of the two modalities. 29
4.3 Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations. 31
4.4 Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments. 33
4.5 Hand-crafted features of the classical baseline. 35
4.6 Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer. 36
4.7 Training settings shared by all neural network models. 37
4.8 Training and validation windows per class after the grouped split. . . 39

List of Abbreviations

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

4 Model Design and Training

This chapter describes the models used for activity recognition and the procedure used to train and evaluate them. It begins with an overview of the experimental design and its relation to the research questions. It then covers the input representations derived from the inertial and acoustic windows of Chapter 3, the single-modality models, the three fusion strategies, and a classical baseline using hand-crafted features. It closes with the training procedure and the evaluation protocol that all models share. The results obtained with these models are presented in Chapter 6, and the conversion of selected models for execution on the wearable device is covered in Chapter 5.

4.1 Overview of the Experimental Design

The experiments compare models that differ in one respect at a time while sharing everything else: the same windows, the same train/validation split, the same input normalisation and the same training settings. This makes differences in accuracy attributable to the model design rather than to differences in the data or in the training procedure.

The models fall into four groups, each linked to one or more of the research questions formulated in Chapter 1:

- **Single-modality models** use either inertial or acoustic data alone. They form the reference point against which the benefit of adding audio is measured (RQ1, RQ3).
- **Fusion models** combine both modalities at different stages of processing: at the input level (early fusion), at the level of learned representations (embedding-level fusion), or at the level of class predictions (decision-level fusion). Their comparison addresses RQ2.
- **A classical baseline** based on statistical features and a random forest shows how much the learned models gain over a conventional approach (RQ3).
- **Reduced model variants** with fewer parameters were designed with the memory and computational limits of the target device in mind. They connect the offline comparison to the deployment study in Chapter 5 (RQ4).

All experiments use the dominant-wrist dataset described in Chapter 3, with 2565 paired windows across nine activity classes. The standalone audio model is the one

Group	Model	Inputs	Fusion stage
Single modality	IMU CNN	inertial	–
Single modality	Audio CNN (GAP, GRU and hybrid pooling)	acoustic	–
Early fusion	Channel concatenation (standard and small)	inertial + acoustic	input
Early fusion	Unified spectrogram	inertial + acoustic	input
Embedding-level fusion	Frozen backbones with a trained fusion head (standard and small)	inertial + acoustic	representation
Decision-level fusion	Weighted average with global or per-class weights (standard and small)	inertial + acoustic	prediction
Classical baseline	Random forest on hand-crafted features	inertial + acoustic	feature

Table 4.1: Overview of the evaluated models.

exception: it additionally uses audio windows from the non-dominant wrist and the waist, as explained in Section 4.3.2.

4.2 Input Representations

4.2.1 Inertial Windows

Each inertial window holds 160 samples of the six channels (three-axis acceleration and three-axis angular velocity) recorded at 80 Hz. Before entering a model, the window is resampled to 200 time steps by linear interpolation along the time axis, so that all inertial models take inputs of size 200×6. The target length of 200 was chosen to match the time resolution of the audio spectrogram, which has 198 frames per window (Section 4.2.2). This allows both modalities to share one time axis in the early fusion model (Section 4.4.1) without coarsening the audio representation, and using the same length in all inertial models keeps their inputs identical across the experiments. Missing or non-finite values, which can remain at the very edges of a recording after decimation, are replaced by zero.

Accelerometer and gyroscope channels have different physical units and value ranges. Each channel is therefore standardised to zero mean and unit variance. The mean and standard deviation of each channel are computed on the training split only and then applied unchanged to the validation data, so that no statistics of the validation set enter the training process. The same statistics are later embedded in the device firmware, where they are applied to the live sensor data (Chapter 5).

4.2.2 Audio Log-Mel Spectrograms

Raw audio windows of 32 000 samples are too long to be used directly as input to a small model. Each window is therefore converted into a log-mel spectrogram, a time-frequency representation that is widely used for sound classification [CITATION]. The conversion consists of three steps:

1. a short-time Fourier transform (STFT) with frames of 25 ms (400 samples), a hop of 10 ms (160 samples) and an FFT length of 512;
2. a projection of the magnitude spectrum onto 40 mel-spaced frequency bands between 20 Hz and 8 kHz [CITATION];
3. the natural logarithm of the band energies, with a small constant (10^{-6}) added to avoid the logarithm of zero.

A two-second window produces a spectrogram of 198 frames × 40 mel bands. This reduces the input size by roughly a factor of four compared with the raw waveform, while keeping the spectral and temporal structure that distinguishes, for example, the continuous whine of a powered screwdriver from the short, broadband impacts of hammering. As with the inertial data, the spectrogram is standardised per mel band using the mean and standard deviation of the training split. Figure 4.1 summarises the feature extraction, and Table 4.2 lists the resulting input representations.

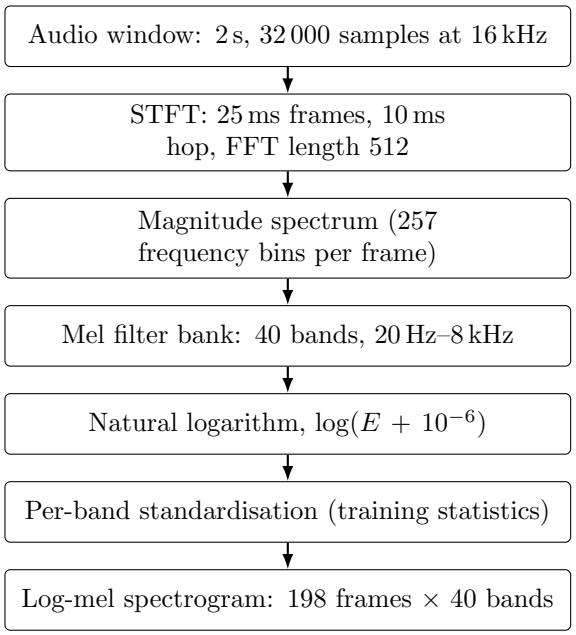


Figure 4.1: Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models.

Modality	Raw window	Model input	Normalisation
Inertial	160×6 at 80 Hz	200×6 (linear resampling)	per channel, training statistics
Audio	32 000 samples at 16 kHz	198×40 log-mel	per mel band, training statistics

Table 4.2: Input representations of the two modalities.

4.3 Single-Modality Models

4.3.1 Inertial Model

The inertial model is a one-dimensional convolutional neural network (1D CNN) that convolves along the time axis of the window. 1D CNNs are a common choice for wearable activity recognition because they learn local motion patterns directly from the raw signal and remain small enough for embedded use [CITATION].

The network has two convolutional blocks. The first block applies 64 filters with a kernel size of 5, followed by batch normalisation and max pooling with a factor of 2. The second block applies 128 filters with a kernel size of 3, followed by batch normalisation. Global average pooling then reduces the temporal dimension to a single feature vector. This vector passes through a dense layer with 64 units, which also serves as the embedding for embedding-level fusion (Section 4.4.2), and through dropout with a rate of 0.3, before a softmax output layer with one unit per class.

4.3.2 Audio Model and Temporal Pooling Variants

The audio model treats the log-mel spectrogram as a single-channel image and processes it with a two-dimensional CNN [CITATION]. It has two convolutional blocks with 32 and 64 filters of size 3×3, each followed by batch normalisation and 2×2 max pooling. The resulting feature map has a size of $49 \times 10 \times 64$ (time $\times$ frequency $\times$ channels).

How this feature map is reduced to a single vector determines which properties of the sound the classifier can use. Three variants were therefore evaluated (Figure 4.2):

- **Global average pooling (GAP).** The feature map is averaged over both time and frequency [CITATION]. This is the standard approach and gives a compact model. However, averaging over time removes the information about how the sound develops within the window. Two activities with a similar average spectrum but a different temporal pattern, such as the regular strikes of a hammer and the steady sound of a screwdriver, can then become hard to separate.
- **Recurrent pooling (GRU).** The feature map is reshaped into a sequence of 49 time steps with 640 features each and passed through a bidirectional

gated recurrent unit (GRU) with 32 units per direction [CITATION]. The GRU summarises the sequence while keeping its temporal order, so the model can use rhythm and envelope information.

- **Hybrid pooling.** Both summaries are computed in parallel and concatenated: the GAP vector, which describes the average timbre of the window, and the GRU output, which describes its temporal structure. The classifier can then rely on whichever information is useful for a given class.

In all three variants, the pooled vector passes through a dense layer with 64 units and dropout with a rate of 0.4, followed by the softmax output.

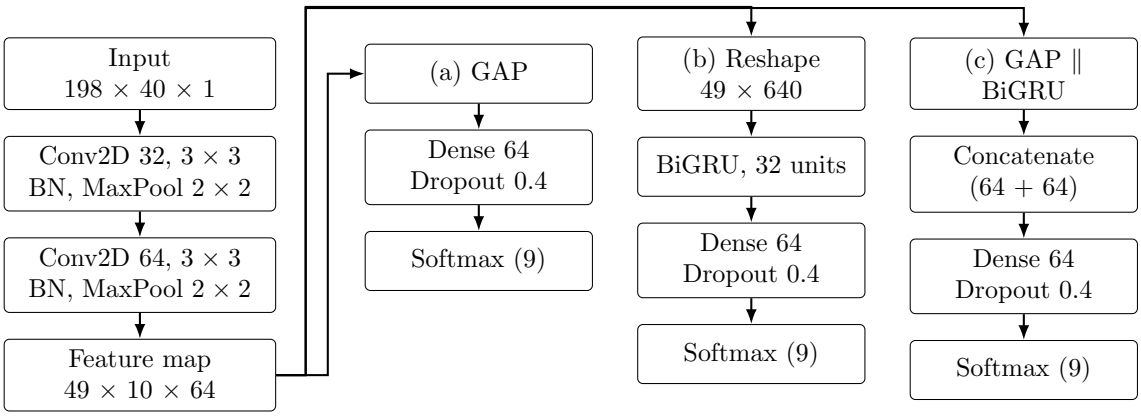


Figure 4.2: Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries.

Because audio content depends less on where the sensor is worn than inertial data does, the standalone audio model is trained on audio windows from three body positions: the dominant wrist, the non-dominant wrist and the waist. This roughly triples the amount of acoustic training data. The pooled audio is used only for the standalone audio model. All fusion models use dominant-wrist audio exclusively, so that every audio window is paired with the inertial window from the same device and time interval.

The pooling variants are compared on the standalone audio model. Within the fusion models, the acoustic component uses the GAP variant. With 23 945 parameters it is about six times smaller than the recurrent variants, and it avoids recurrent layers, whose sequential computation is costly on the target microcontroller. This keeps all fusion models within reach of the deployment study in Chapter 5. The performance of the three variants is reported in Chapter 6. Table 4.3 summarises both single-modality architectures.

Layer	IMU CNN	Audio CNN
Input	200×6	$198 \times 40 \times 1$
Block 1	Conv1D 64, kernel 5, BN, MaxPool 2	Conv2D 32, 3×3, BN, MaxPool 2×2
Block 2	Conv1D 128, kernel 3, BN	Conv2D 64, 3×3, BN, MaxPool 2×2
Pooling	Global average	GAP / BiGRU(32) / GAP + BiGRU
Head	Dense 64, Dropout 0.3	Dense 64, Dropout 0.4
Output	Softmax (9 classes)	Softmax (9 classes)
Parameters	36 297	23 945 (GAP) / 153 353 (GRU) / 157 449 (hybrid)

Table 4.3: Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations.

4.4 Fusion Architectures

The three fusion strategies differ in the point along the processing chain at which the two modalities are combined [CITATION]. Figure 4.3 gives an overview.

4.4.1 Early Fusion

Early fusion combines the two modalities before any learning takes place, so that a single network learns from both signals jointly. Two forms were implemented.

Channel concatenation. The log-mel spectrogram (198×40) is resampled along its time axis to 200 frames, matching the length of the inertial window. The two inputs are then concatenated along the feature axis, giving a 200×46 input with 6 inertial and 40 acoustic channels. Each of the 46 channels is standardised with its own training statistics. This step matters more here than in the single-modality models, because raw inertial values and log-mel energies lie on very different scales. The fused input is processed by a 1D CNN with the same structure as the inertial model (Section 4.3.1).

Unified spectrogram. Here the inertial data are also converted into spectrograms, so that both modalities share one image-like representation. A short-time Fourier transform with a frame length of 20 samples, a hop of 2 samples and an FFT length of 32 is applied to each of the six inertial axes. The log magnitudes are resized to the 198×40 grid of the audio spectrogram by bilinear interpolation. The six inertial spectrograms and the audio spectrogram are stacked as the seven channels of a single input and processed by the 2D audio CNN with global average pooling.

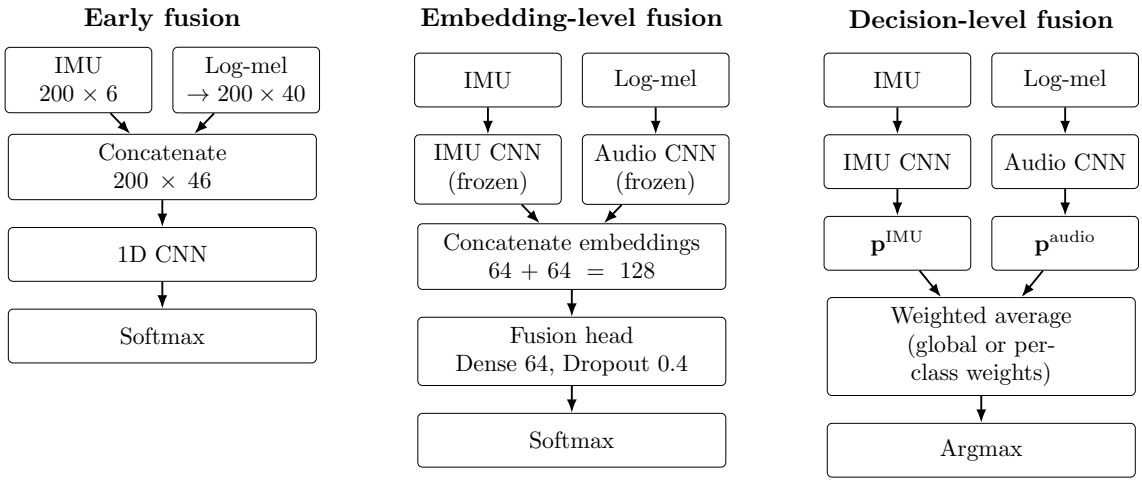


Figure 4.3: The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models.

4.4.2 Embedding-Level Fusion

Embedding-level fusion first trains the inertial and the acoustic model separately, as described in Section 4.3. Their weights are then frozen, and for each window the outputs of the 64-unit dense layer are taken as embeddings. The two 64-dimensional embeddings are concatenated into a 128-dimensional vector, which is the input to a small fusion head: a dense layer with 64 units, dropout with a rate of 0.4 and a softmax output (8841 parameters). Only the fusion head is trained in this second stage. It can learn how the two modalities relate to each other without changing the feature extraction that each backbone has already learned.

4.4.3 Decision-Level Fusion

Decision-level fusion combines the class probabilities predicted by the two independently trained models [CITATION]. For a window x, the fused probability of class c is a weighted average

$$p_c(x) = w \cdot p_c^{\text{IMU}}(x) + (1 - w) \cdot p_c^{\text{audio}}(x), \qquad w \in [0, 1], \tag{4.1}$$

and the predicted class is the one with the highest fused probability. The weight w is selected by a grid search over the values $0, 0.05, \ldots, 1$, keeping the value that gives the highest accuracy on the validation data. Compared with a trained fusion head, this method has only a single free parameter and is therefore less prone to overfitting on a small dataset.

A single weight, however, assumes that the relative reliability of the two modalities is the same for every class. This assumption does not hold for the present activity

set. For some activities the motion pattern is distinctive while the sound is not; standing idle and measuring, for example, produce hardly any sound. For others the opposite is true, and the powered screwdriver is the clearest example of an activity with a distinctive acoustic signature. The method was therefore extended to one weight per class, which gives a weight vector $\mathbf{w} = (w_1, \ldots, w_C)$ and

$$p_c(x) = w_c \cdot p_c^{\text{IMU}}(x) + (1 - w_c) \cdot p_c^{\text{audio}}(x). \tag{4.2}$$

The only difference to Equation 4.1 is that the weight now carries the index c. When the fused model computes the score of a particular class, it uses the weight of that class, and it can therefore rely on whichever sensor is more informative for that class. A value of w_c close to 1 means that the score of class c is determined mainly by the inertial model, and a value close to 0 means that it is determined mainly by the acoustic model.

A short example illustrates the effect. Consider a window of the class `measuring`, for which the two models produce the probabilities shown in Table 4.4; only three classes are shown for readability. The inertial model recognises the activity, whereas the acoustic model hears almost silence and leans towards `idle`. With a global weight of $w = 0.5$, the correct class wins only by a small margin. If the weight of `measuring` is raised to $w_{\text{measuring}} = 0.9$, its score is taken almost entirely from the more reliable inertial model, and the margin increases clearly. The score of `idle` does not change, because its own weight has not changed.

	`measuring`	`idle`	`hammering`
$\mathbf{p}^{\text{IMU}}$	0.70	0.25	0.05
$\mathbf{p}^{\text{audio}}$	0.20	0.60	0.20
Global weight, $w = 0.5$	**0.45**	0.425	0.125
Per-class weight, $w_{\text{measuring}} = 0.9$, others 0.5	**0.65**	0.425	0.125

Table 4.4: Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments.

The nine weights cannot be found by simply trying every combination. With 21 candidate values per weight, this would require $21^9 \approx 8 \cdot 10^{11}$ evaluations. Instead, the weights are determined by coordinate ascent, which adjusts one weight at a time while keeping the others fixed:

1. **Initialisation.** All nine class weights are set to the best global weight from Equation 4.1. The search therefore starts from the result of global fusion and cannot end with a lower validation accuracy.

2. **Adjusting one class.** For the first class, its weight is set in turn to each of the 21 values $0, 0.05, \ldots, 1$, while the weights of all other classes stay unchanged.

For each value, all validation windows are fused with Equation 4.2 and the overall accuracy is computed. The weight is replaced only if one of the values gives a strictly higher accuracy than the current setting.

3. **All classes.** Step 2 is repeated for the second class, the third class and so on, until every class has been adjusted once. This is one pass.

4. **Second pass.** Because changing one weight can slightly shift the best value of another, the whole pass is repeated once more.

In total, the search evaluates $2 \times 9 \times 21 = 378$ weight settings, which takes only a fraction of a second. Two passes are sufficient because the classes interact only weakly. Changing w_c affects only those windows whose decision was already close between class c and another class; for most windows one class leads clearly, and a single weight cannot change the outcome. The resulting weights can be read as the degree of trust that the fused model places in each modality for each class, which also makes them useful for interpreting the results in Chapter 6. Figure 4.4 summarises the procedure.

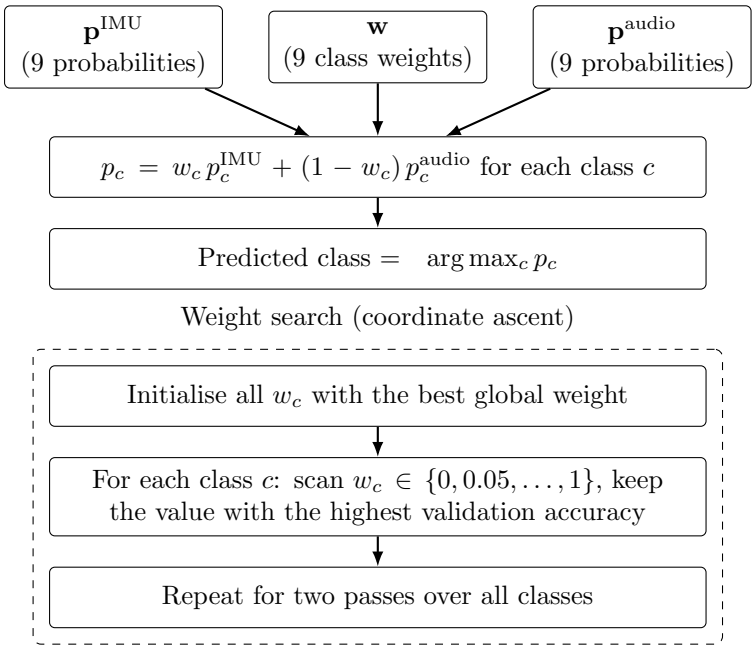


Figure 4.4: Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom).

4.5 Classical Baseline

To place the learned models in context, a conventional approach based on hand-crafted features was also evaluated. From each inertial window, six statistics are

computed per channel: mean, standard deviation, minimum, maximum, root mean square and zero-crossing rate, which gives 36 inertial features. From each log-mel spectrogram, the mean and standard deviation of every mel band over time are computed, which gives 80 acoustic features. The two feature sets are concatenated into a 116-dimensional vector (Table 4.5), so the baseline also uses feature-level fusion.

A random forest with 300 trees is trained on these vectors, with balanced class weights to account for the class imbalance [CITATION]. Random forests need no feature scaling and give robust results on small tabular datasets, which makes them a common reference point in activity recognition [CITATION].

Modality	**Features**	**Count**
Inertial	mean, standard deviation, minimum, maximum, RMS and zero-crossing rate for each of the 6 channels	36
Audio	mean and standard deviation over time for each of the 40 mel bands	80
Total		**116**

Table 4.5: Hand-crafted features of the classical baseline.

4.6 Deployment-Oriented Model Variants

The models described so far were designed for offline evaluation. On the target microcontroller, however, inference time and memory usage are strict limits. The TensorFlow Lite Micro build for this platform executes convolutions with unaccelerated reference kernels, so their cost grows directly with the number of filters and with the size of the feature maps (Chapter 5). Reduced variants were therefore created that keep the overall architecture but use fewer filters:

- **Small early fusion model.** The structure of the channel-concatenation model with 32 and 64 instead of 64 and 128 filters. This reduces the number of multiply-accumulate operations, and with it the inference time, by roughly a factor of four.
- **Small inertial backbone.** A 1D CNN with 32 and 64 filters.
- **Small audio backbone.** A 2D CNN with 16 and 32 filters. Its convolutions use strides of 4 and 2 instead of separate pooling layers, so the downsampling happens within the convolution itself. As a result, a feature map at the full 198×40 resolution never has to be held in memory, which would otherwise take up a large part of the device's working memory.

The small backbones are used for the small embedding-level and decision-level fusion models. Because these models have less capacity, three regularisation measures were added to reduce overfitting: L2 weight decay (10^{-4}) on all learned layers, higher dropout rates (0.4 and 0.5), and label smoothing of 0.1 in the loss function [CITATION].

In addition, the training data of the small backbones are augmented. One augmented copy is generated for each training window, so that the training set doubles in size; the validation data are never augmented.

- **Inertial windows** receive a small random three-dimensional rotation of up to $\pm 15°$, applied jointly to the accelerometer and gyroscope axes to mimic small changes in how the device sits on the wrist, together with per-channel scaling (factor 0.9–1.1), additive noise (3 % of the channel's standard deviation) and a mild time warp (speed factor 0.9–1.1) [CITATION].
- **Spectrograms** receive a small circular time shift of up to ± 10 frames and SpecAugment-style masking with two random time masks of up to 20 frames and two frequency masks of up to 6 bands, filled with the mean value of the spectrogram [CITATION].

Table 4.6 compares the standard and the small variants.

Model	Standard	Small
Early fusion (concatenation)	Conv1D 64 / 128; 49 097 parameters	Conv1D 32 / 64; 18 729 parameters
IMU backbone	Conv1D 64 / 128; 36 297 parameters	Conv1D 32 / 64, L2, dropout 0.4; 12 329 parameters
Audio backbone	Conv2D 32 / 64 with max pooling; 23 945 parameters	Conv2D 16 / 32 with strides 4 / 2, L2, dropout 0.5; 7 689 parameters
Label smoothing	–	0.1 (backbones)
Augmentation	–	one augmented copy per training window (backbones)

Table 4.6: Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer.

4.7 Training Procedure

4.7.1 Software and Hardware Environment

All models were implemented in Python 3.10 with TensorFlow 2.10 and Keras [CITATION]. Training ran on an NVIDIA GeForce RTX 4070 Laptop GPU through

the DirectML backend for Windows. This backend supports TensorFlow only up to version 2.10, which fixed the framework version for the whole project. Two parts of the pipeline could not run on the GPU with this backend and had to be adapted: the short-time Fourier transform used for the spectrograms was moved to the CPU, and the fused recurrent kernel of the GRU layer was replaced by an unrolled implementation. Both changes affect only where and how the computation is executed, not its result. Data preparation used NumPy, pandas and librosa [CITATION], and the classical baseline used scikit-learn [CITATION].

4.7.2 Training Settings

All neural networks were trained with the settings listed in Table 4.7. The class imbalance described in Section 3.8 is handled through class weights in the loss function. The weight of each class is inversely proportional to its frequency in the training split, so that errors on rare classes count more than errors on frequent ones [CITATION]. The training data themselves are left unchanged, without oversampling or undersampling.

Setting	**Value**
Optimiser	Adam [CITATION]
Learning rate	10^{-3}, constant
Batch size	32
Loss	Categorical cross-entropy (label smoothing 0.1 for the small backbones)
Class weighting	Inverse class frequency (“balanced”)
Epochs	150 for backbones and early-fusion models; at most 80 for the fusion head
Model selection	Weights of the epoch with the highest validation accuracy
Fusion head	Early stopping (patience 15), learning rate halved on plateau (patience 6, minimum 10^{-5})

Table 4.7: Training settings shared by all neural network models.

The backbones and the early-fusion models are trained for a fixed 150 epochs at a constant learning rate. On a small validation set, the validation accuracy can fluctuate noticeably from one epoch to the next, so the weights at the end of training are not necessarily the best ones. After training, each model is therefore set back to the weights of the epoch with the highest validation accuracy.

In the Keras version used here, the standard early-stopping callback restores the best weights only when it actually stops training early. If training runs the full number of epochs, the weights of the last epoch are kept without any warning. A small custom callback was therefore written that stores the best weights during training and restores them at the end. The effect of this choice is examined in

Chapter 6. Because the best epoch is selected on the validation data, the procedure introduces a slight optimistic bias into the reported results, a point taken up in the discussion of limitations in Chapter 7.

4.8 Evaluation Protocol

4.8.1 Grouped Train/Validation Split

The windows were created with 50 % overlap (Section 3.7), so neighbouring windows share half of their samples. A random split would place such neighbours on both sides of the train/validation boundary. The model would then be evaluated on data that it had partly seen during training, and the measured accuracy would overestimate its performance on new data [CITATION].

To prevent this, the windows are first grouped into activity instances. Consecutive windows with the same label belong to the same group as long as their start times are at most 1.5 s apart, that is, 1.5 times the window stride. Consecutive windows of one uninterrupted activity occurrence start exactly 1 s apart, so the additional 0.5 s serves as a margin: small irregularities in the window start times do not split one occurrence into several groups, while occurrences separated by a pause or by another activity, which leave a gap of at least one further stride, are still kept apart. The 2565 windows form 125 such groups. The split is then made per class: the groups of a class are shuffled, and whole groups are assigned to the validation set until it holds at least 20 % of that class's windows. The remaining groups form the training set. Every activity instance therefore lies entirely in one of the two sets, and a fixed random seed makes the split identical for all models, so that their results can be compared directly. Figure 4.5 illustrates the principle.

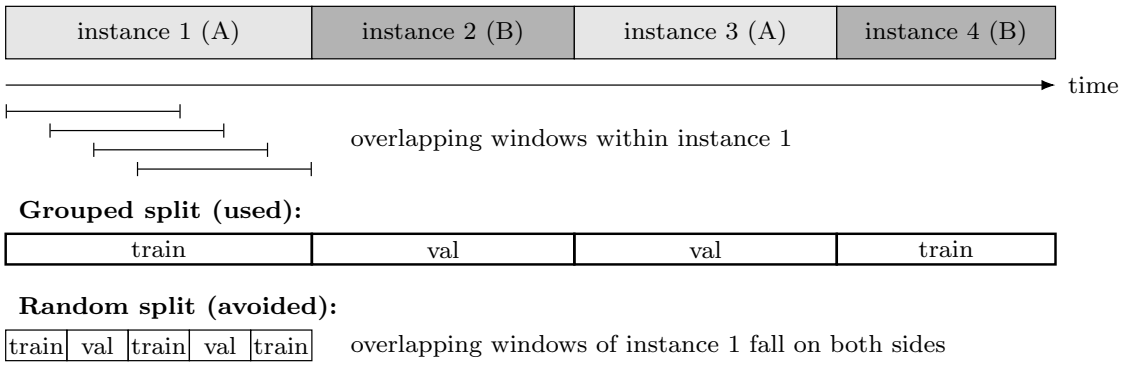


Figure 4.5: Grouped splitting. Overlapping windows are grouped into activity instances, and whole instances are assigned to either the training or the validation set. A random split would distribute the overlapping windows of one instance across both sets.

Table 4.8 shows the resulting split. Because only whole groups can be moved, the validation share of a class is not exactly 20 %. It exceeds the target considerably

when a class consists of a few long instances, as is the case for `walk`, `measuring` and `place_container`, where a single group already contains a large part of the class. Overall, 946 windows (37 %) fall into the validation set and 1619 windows (63 %) into the training set. For `place_container`, only 23 windows remain for training. This uneven split is a consequence of keeping activity instances intact, and its effect on the results is discussed in Chapter 7.

Class	Training	Validation	Total	Validation share
assemble_parts	130	36	166	22 %
carry_container	175	45	220	20 %
hammering	170	80	250	32 %
idle	187	120	307	39 %
insert_screws	185	61	246	25 %
measuring	159	161	320	50 %
place_container	23	51	74	69 %
screw_tightening	454	142	596	24 %
walk	136	250	386	65 %
Total	**1619**	**946**	**2565**	**37 %**

Table 4.8: Training and validation windows per class after the grouped split.

4.8.2 Metrics

Overall accuracy, the share of correctly classified validation windows, is used as the main measure for comparing the models. Because the classes are imbalanced, accuracy alone can hide poor performance on small classes. Precision, recall and F1-score are therefore also reported for each class, together with the macro-averaged F1-score, which gives every class the same weight [CITATION]. Confusion matrices are used to identify which activities are mistaken for one another. Unless stated otherwise, all results are reported on the validation set of the grouped split and come from a single training run per model.

4.9 Chapter Summary

This chapter introduced the models compared in this thesis. Inertial windows are processed as 200×6 time series by a 1D CNN, and audio windows as 198×40 log-mel spectrograms by a 2D CNN, for which three pooling variants were developed to make use of the temporal structure of the sounds. The two modalities are combined at the input level (channel concatenation and unified spectrogram), at the level of learned embeddings, and at the level of class probabilities, where a per-class weighting reflects the different reliability of the two modalities across activities. A random forest on hand-crafted features serves as a classical baseline, and reduced,

regularised variants prepare the deployment on the device. All models share the same input normalisation, class weighting, training settings, best-epoch selection and grouped train/validation split. Chapter 5 describes how selected models were converted for execution on the wearable device, and Chapter 6 presents the results.

5 On-Device Deployment

6 Results and Evaluation

7 Discussion

8 Conclusion